

Chapitre II

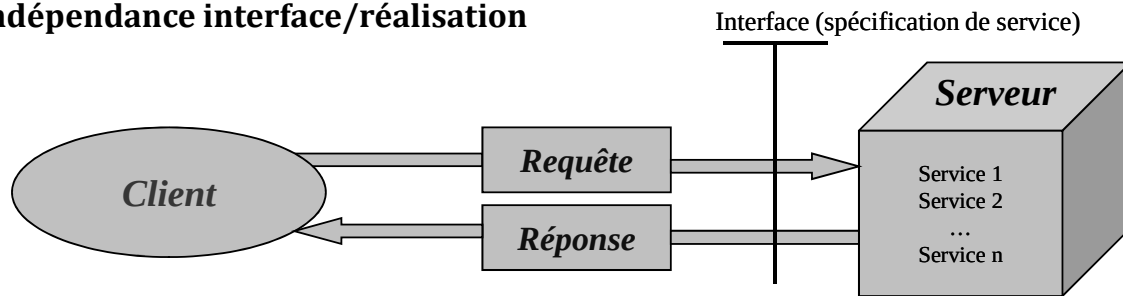
INTRODUCTION AU MODÈLE CLIENT/SERVEUR (Classique)

PLAN DU CHAPITRE

- ① Modèle Client/Serveur
 - ⇒ Définitions
 - ⇒ Caractéristiques
 - ⇒ Intérêt
- ② Conception d'applications Client/Serveur
- ③ Appel de procédure à distance (RPC)
 - ⇒ Principe de réalisation
 - ⇒ Génération des talons
 - ↳ Utilisation de ***rpcgen***
 - ⇒ Problèmes de mise en œuvre

1. MODÈLE CLIENT/SERVEUR: Définitions (1/2)

- ❑ Approche universellement adoptée pour structurer les applications coopératives.
 - ⇒ Modèle de base.
- ❑ **Client**: émet des requêtes (demande de services) vers le serveur grâce à son adresse et son port
- ❑ **Serveur**: exécute les requêtes émises par les clients (réponse aux demandes, réalisation de services)
 - ⇒ Nécessite souvent des privilèges système
- ❑ **Indépendance interface/réalisation**



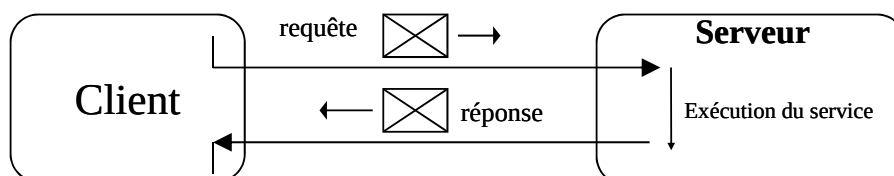
Modèle Client / Serveur

3

a1

MODÈLE CLIENT/SERVEUR: Définitions (2/2)

- ❑ **Communication par messages** (pas partage de données, mémoires ou fichiers)
 - ⇒ **Requête**: paramètres d'appel, spécification de services requis
 - ⇒ **Réponse**: résultats, indicateur éventuel d'exécution ou d'erreur
 - ⇒ **Communication synchrone**:
 - ↪ Le client est l'initiateur du dialogue
 - ↪ Le client est bloqué en attente de la réponse



Modèle Client / Serveur

4

MODÈLE CLIENT/SERVEUR: Intérêt

❑ STRUCTURATION

- ⇒ Fonctions bien identifiées
- ⇒ Séparation interface du service-réalisation
- ⇒ Client et serveur peuvent être modifiés (remplacés) indépendamment

❑ PROTECTION

- ⇒ Client et serveur s'exécutent dans des domaines de protection différents

❑ PARTAGE DES RESSOURCES

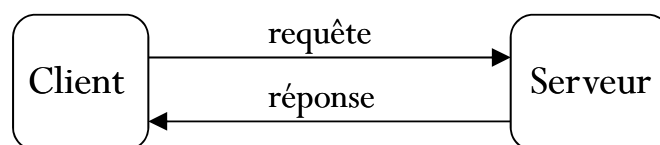
- ⇒ Le serveur peut être partagé entre plusieurs clients
- ➔ **Ces considérations sont indépendantes de la répartition**

MODÈLE CLIENT/SERVEUR: Caractéristiques

- ❑ **Service:** le serveur est le fournisseur et le client est le consommateur de services.
- ❑ **Partage des ressources:** un serveur traite plusieurs clients en même temps et contrôle leurs accès aux ressources.
- ❑ **Localisation:** le logiciel client/serveur masque aux utilisateurs la localisation du serveur.
- ❑ **Hétérogénéité:** le logiciel client/serveur est indépendant des plateformes matérielles et logicielles
- ❑ **Redimensionnement:** il est possible d'ajouter et de retirer des clients. Il est possible de faire évoluer les serveurs
- ❑ **Intégrité:** les données du serveur sont gérées de façon centralisée. Les clients restent individuels et indépendants
- ❑ **Souplesse et adaptabilité:** on peut modifier le module serveur sans toucher au module client. La réciproque est vraie

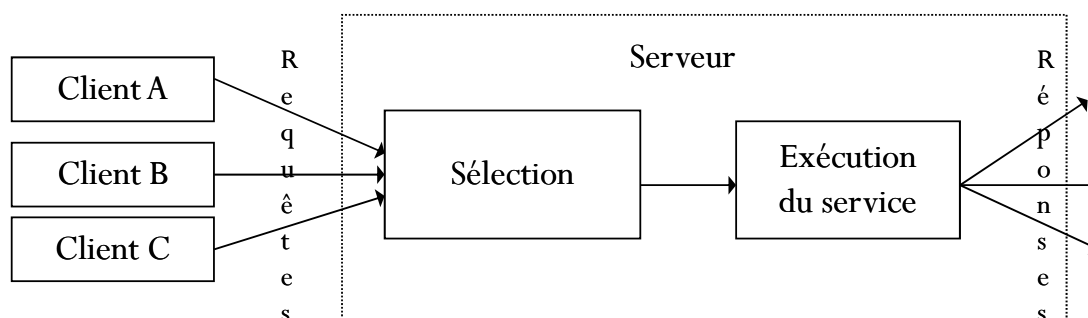
MODÈLE CLIENT/SERVEUR: Partage du serveur

❑ Vu du client



❑ Vu du serveur

- ⇒ Gestion des requêtes (priorité)
- ⇒ Exécution du service (séquentiel, concurrent)
- ⇒ Mémorisation ou non de l'état du client



MODÈLE CLIENT/SERVEUR: Exemples (1/2)

- ☐ Serveur de fichiers (NFS, AFS)
- ☐ Serveur d'impression (lpd)
- ☐ Serveur de noms de domaines (DNS)
- ☐ Serveur d'applications
- ☐ Serveur d'objets:
 - ⇒ Invocation d'objets distants (RMI)
 - ⇒ C/S à objets repartis (CORBA, DCOM)

MODÈLE CLIENT/SERVEUR: Exemples (2/2)

- ☐ Serveur de base de données:
 - ⇒ Accès distant à une BD
 - ⇒ Les requêtes (commandes SQL).
 - ⇒ Exécution par le serveur de la BD (ou un serveur d'applications)
 - ⇒ ODBC –Open DB connection
- ☐ Serveur Transactionnels:
 - ⇒ OLTP –On Line Transaction Processing

MODÈLE CLIENT/SERVEUR: Mise en œuvre

❑ Gestion de l'état

- ⇒ Information d'état maintenue par le serveur sur les interactions avec le client.
- ⇒ État du serveur (persistant ou non)
- ⇒ État du client (mémorisé ou non par le serveur)

❑ Utilisation d'un service de communication par messages

- ⇒ Exemples: sockets Unix
 - ↳ Mode connecté: messages sur liaison virtuelle (TCP)
 - ↳ Mode non connecté: messages indépendants (UDP)

❑ Gestion de l'exécution sur le serveur

- ⇒ un ou plusieurs processus
- ⇒ Pool fixe ou création à demande

Serveur sans données rémanentes (persistantes)

❑ L'exécution du service utilise uniquement les paramètres d'entrée (seulement)

- ⇒ Situation « idéale »
- ⇒ Produire uniquement des paramètres résultats
- ⇒ Pas de modification de données rémanentes sur le site serveur

❑ Solution très favorable:

- ⇒ Pour la tolérance aux pannes (pas d'état à sauvegarder ou à restaurer)
- ⇒ Pour le contrôle de concurrence

❑ Exemple:

- ⇒ Calcul d'une fonction mathématique ($\sin(x)$)

Serveur avec données rémanentes

❑ Les exécutions successives manipulent des données rémanentes (persistantes)

- ⇒ Modification du contexte d'exécution sur le site distant
- ⇒ Problème de contrôle de la concurrence
- ⇒ Difficultés en cas de pannes en cours d'exécution

❑ Exemples:

- ⇒ Données persistantes: la structure de données manipulées par les méthodes d'un objet déclaré persistant
- ⇒ Serveur de fichiers répartis
 - ↳ Opération d'écriture
 - ↳ Verrouillage pour des données concurrentes
- ⇒ Serveur de base de données

Serveur sans état contre Serveur avec état

❑ Serveur sans état (stateless)

- ⇒ Ne mémorise pas les informations d'état (relatives aux appels en cours)
- ⇒ Les appels successifs sont indépendants
 - ↳ Possibilité de modification des données globales (pas de lien avec celles qui l'ont précédé)
 - ↳ L'ordre des requêtes ne peut pas être préservé par le serveur
- ⇒ Exemples:
 - ↳ Écriture du n^{ième} article d'un fichier (n fourni dans la requête)
 - ↳ NFS de SUN (système de gestion de fichiers répartis sans état)

❑ Serveur avec état (statefull)

- ⇒ Les appels successifs s'exécutent en file (d'un état laissé par les appels antérieurs)
 - ↳ La préservation de l'ordre des requêtes est indispensable
- ⇒ Exemple:
 - ↳ Lecture d'un enregistrement d'un fichier en accès séquentiel (dépendant du pointeur courant)

Serveur sans état contre Serveur avec état

	Avantages	Inconvénients
Serveur avec état	▪ Exécution plus rapide (les informations nécessaires sont connues par le serveur). ▪ Moins d'informations à transférer sur le réseau. ▪ Gestion de la concurrence.	Pas de tolérance aux pannes.
Serveur sans état	Tolérant aux pannes	Plus lent à l'exécution. Les paramètres doivent être interprétés à chaque fois, aucune anticipation n'est possible.

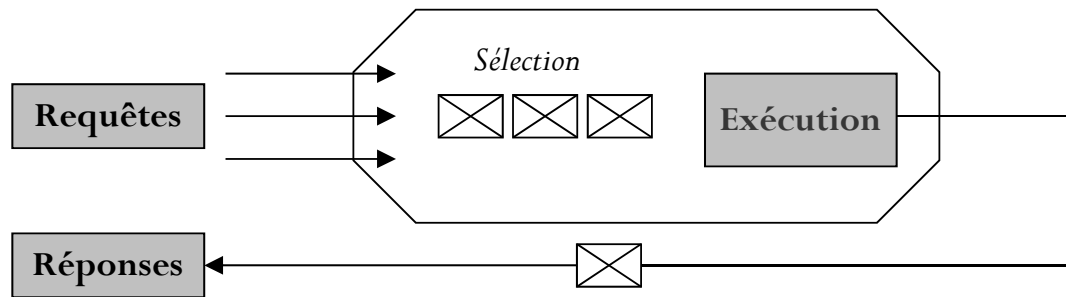
MODÈLE CLIENT/SERVEUR: Gestion des processus

❑ Client et serveur exécutent des processus distincts

- ⇒ Le client est suspendu lors de l'exécution de la requête (appel synchrone)
- ⇒ Éventuellement, plusieurs requêtes peuvent être traitées concurremment par le serveur
 - ↳ Parallélisme réel (multiprocesseurs)
 - ↳ Pseudo-parallélisme --Concurrence
- ⇒ Plusieurs formes de concurrence
 - ↳ Plusieurs processus (une mémoire virtuelle par processus)
 - ↳ Plusieurs processus légers (threads) dans le même espace virtuel (contexte restreint: pile, mot d'état, registres)

GESTION DES PROCESSUS DANS LE SERVEUR (1/3)

Serveur Itératif



- ☐ Processus serveur unique
- ☐ File d'attente pour l'exécution des requêtes

```

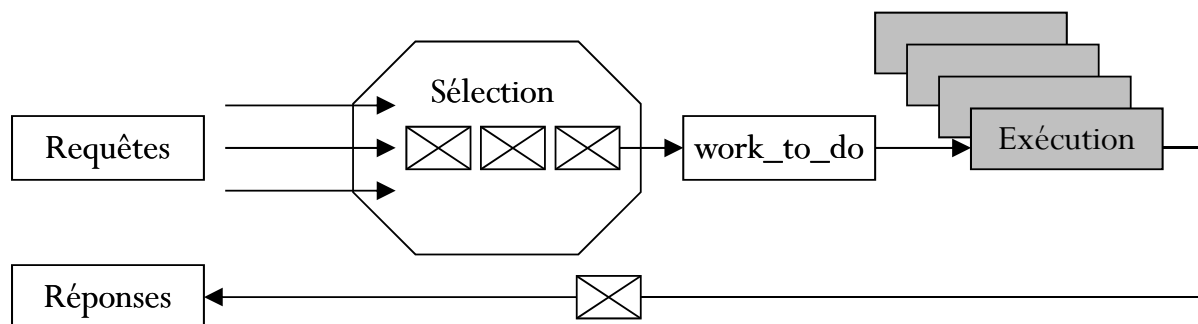
While (true)
{
    receive (client_id, message);
    extract (message, service_id, params);
    exec_service[service_id](params, results);
    send (clients_id, results);
};
    
```

Modèle Client / Serveur

16

GESTION DES PROCESSUS DANS LE SERVEUR (2/3)

Pool fixe -- Sélection



Processus veilleur

```

While (true)
{
    receive (client_id, message);
    extract (message, service_id, params);
    work_to_do.put (clients_id, service_id,
    params);
};
    
```

Pool d'exécutants

```

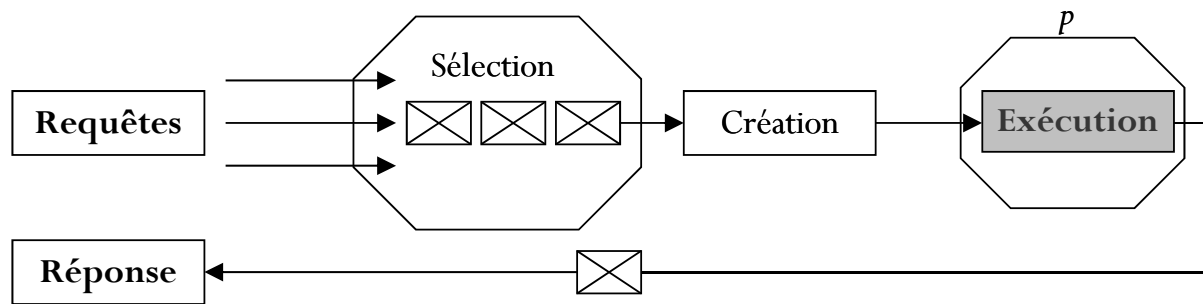
While (true)
{
    work_to_do.put(clients_id,
    service_id, params);
    exec_service[service_id](params, results);
    send (clients_id, results);
};
    
```

Modèle Client / Serveur

17

GESTION DES PROCESSUS DANS LE SERVEUR (3/3)

Création à la demande



Processus veilleur

```

While (true)
{
    receive (client_id, message);
    extract (message, service_id, params);
    p=create_thread (clients_id, service_id,
                    params);
};

```

Création dynamique des exécutants

```

Programme de p
exec_service[service_id](params,
                           results);
send (clients_id, results);
exit();

```

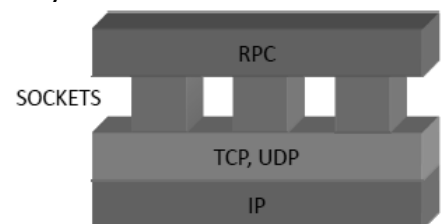
Mise en œuvre du schéma Client/Serveur

❑ Par des opérations de « bas niveau »

- ⇒ Utilisation de primitives du système de communication
- ⇒ Exemple: l'API Unix Socket
 - ↳ Mode connecté: messages sur liaison virtuelle (TCP)
 - ↳ Mode non connecté: messages indépendants (UDP)

❑ Par des opérations de « haut niveau »

- ⇒ Utilisation d'un médiateur (middleware) spécialisé
- ⇒ Contexte: langage de programmation
 - ↳ Appel de procédure à distance (RPC)
 - ↳ Appel de méthodes à distance (RMI), création d'objet à distance



Application Répartie client/serveur?

❑ Quelle découpe de l'application répartie (C/S)?

❑ Une découpe naturelle d'une application (C/S):

⇒ Présentation :

↳ Interface homme-machine (textuelles/graphiques), interactions, entrée des données, validation, etc.

⇒ Logique applicative:

↳ Traitements

↳ Applications "métiers"

⇒ Gestion des données:

↳ Stockage et accès aux données

✓ **Découpe en RPC:**

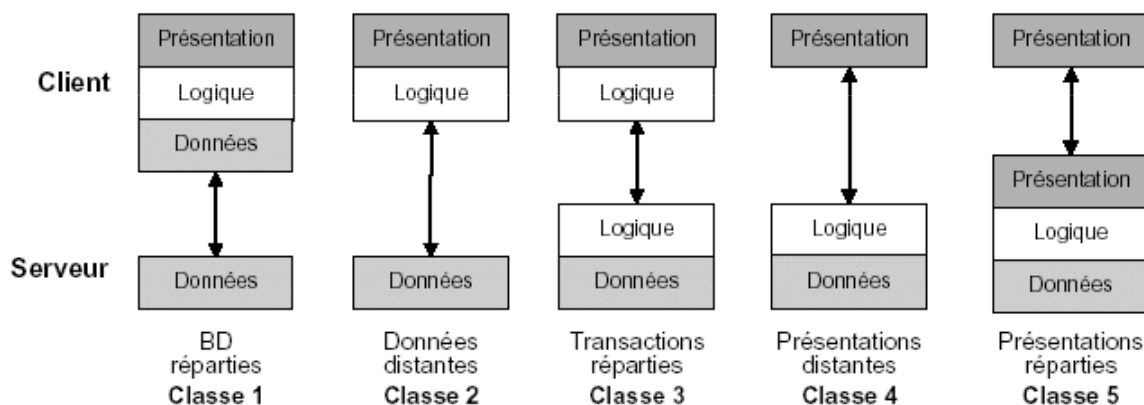
⇒ Conception orientée application

⇒ Communication prise en compte implicitement

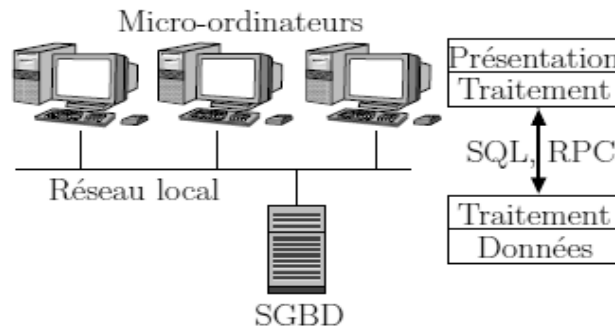
Les niveaux du modèle C/S

❑ **Niveau** = découpage logique d'une application entre des clients et des serveurs, répartition des charges

❑ Tout tient à la façon dont l'application C/S est partagée en unités fonctionnelles que l'on peut alors assigner au client ou à (aux) serveur(s)

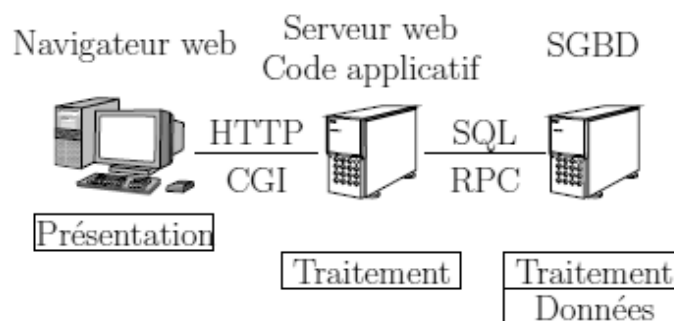


Modèle Client/Serveur 2-tiers



- ❑ Système à 2 niveaux: 1 gros serveur (mainframes) + n terminaux légers connectés au serveur.
- ☺ Pas de duplication de données (état global observable), gestion simple de la cohérence et de l'intégrité, et maîtrise globale des traitements
- ☹ Modèle trop rigide qui n'assure pas l'évolutivité, souvent solutions propriétaires fermées et économiquement trop coûteux

Modèle Client/Serveur 3-tiers (1/2)



- ❑ Systèmes à 3 niveaux: 1 serveur de données et 1 serveur de traitement + n PC avec IHM « évoluées ».
- ❑ Les moniteurs transactionnels, les objets répartis et le Web sont des exemples de systèmes C/S à 3 niveaux, alors que les serveurs de fichiers et les serveurs de BD's à procédures stockées sont des exemple de systèmes C/S à 2 niveaux

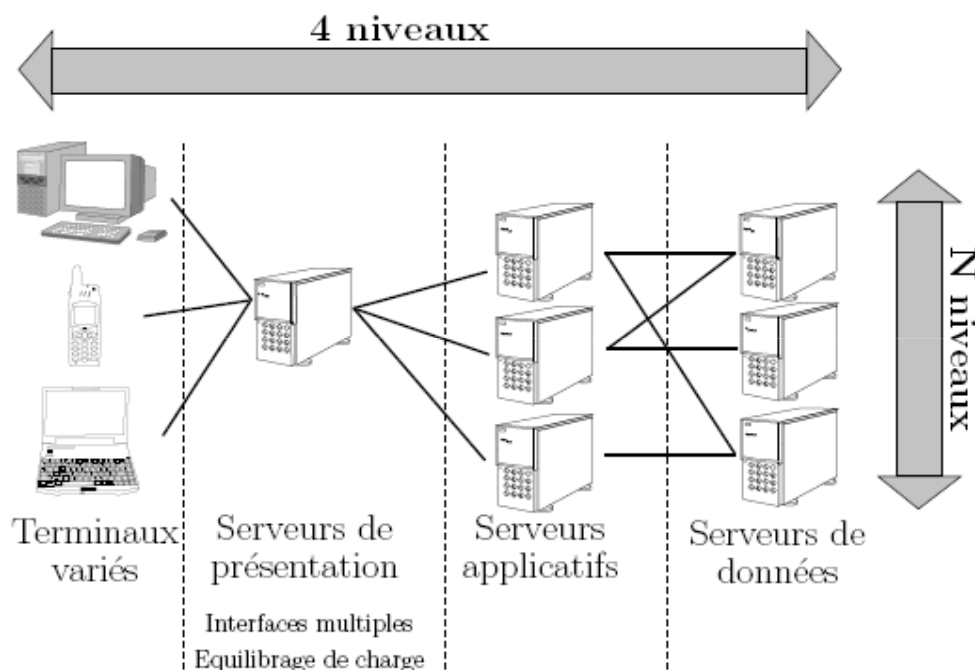
Modèle Client/Serveur à 3 tiers (2/2)

☺ Avantages du modèle C/S 3-tiers:

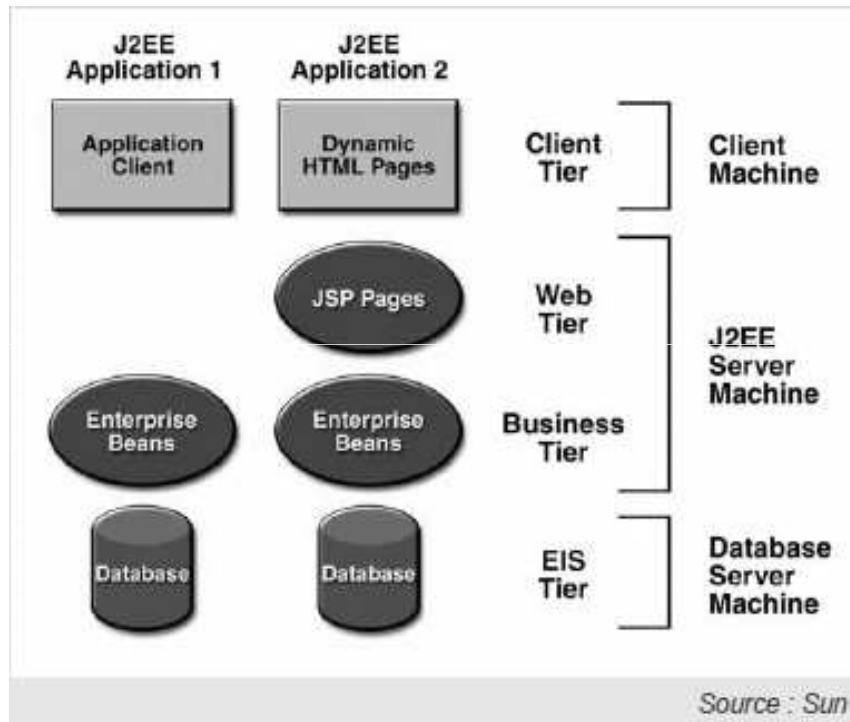
- ⇒ Economiquement moins chère;
- ⇒ Une plus grande flexibilité/souplesse/plus évolutive.
- ⇒ Plus grande sécurité (la sécurité peut être définie pour chaque service)
- ⇒ De meilleures répartition de la charge entre différents serveur d'application et donc meilleures performances
- ⇒ Peut intégrer des données provenant de sources multiples
- ⇒ Prise en compte de l'hétérogénéité des plateformes (serveurs, clients, langages, etc.)

☹ L'administration et la mise en œuvre sont plus compliquées

Modèle C/S N-tiers



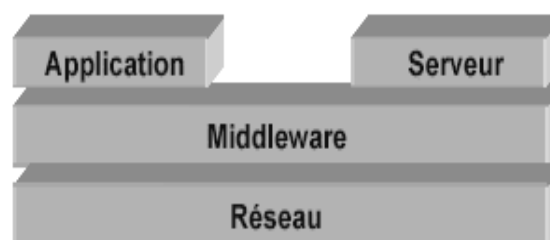
Exemple de Modèle C/S N-tiers –J2EE



Modèle Client / Serveur

26

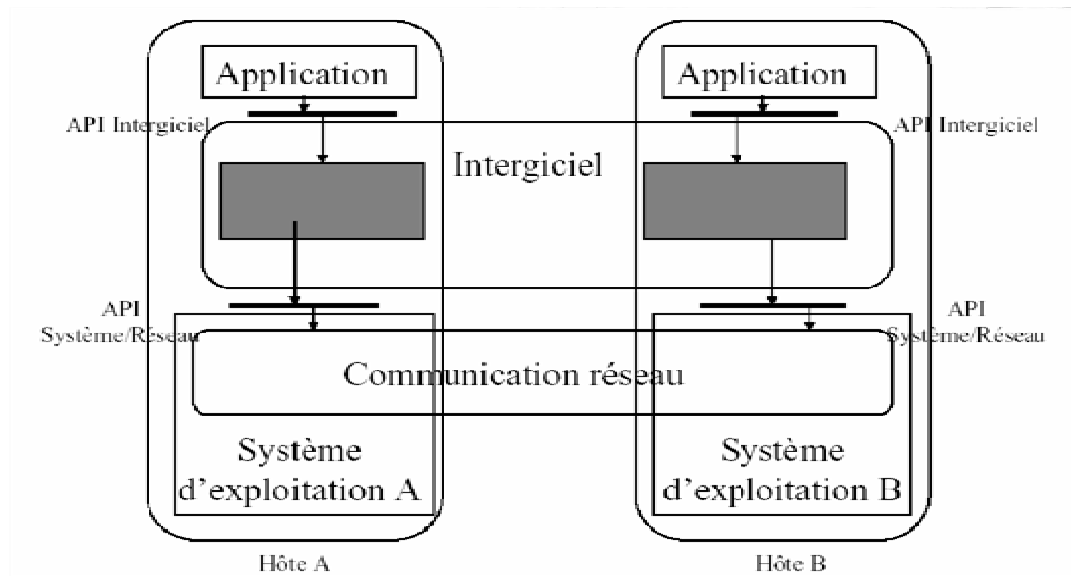
Le middleware: Rôle (1/2)



- ☐ Établi le lien avec la couche transport
- ☐ Procédure de connexion/déconnexion
- ☐ Réalise la synchronisation entre client et serveur
- ☐ Préparation/communication des requêtes
- ☐ Définit le format des données échangées
- ☐ Gestion de caches (requêtes et résultats)

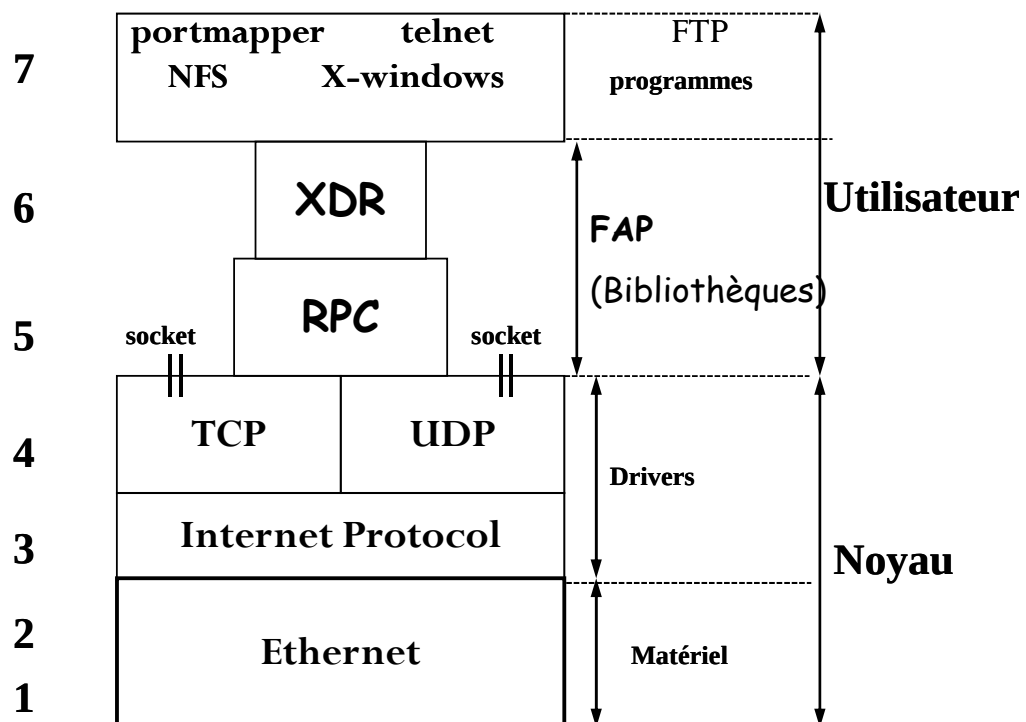
Modèle Client / Serveur

Le middleware: Rôle (2/2)



Modèle Client / Serveur

Pile protocolaire d'une machine Unix



Modèle Client / Serveur

Mise en œuvre du schéma Client/Serveur: RPC

- ❑ Appel de procédure à distance --APD (Remote Procedure Call --RPC)
- ❑ Outil de haut niveau pour la réalisation du schéma client/serveur
- ❑ Idée de Birell & Nelson (1984)
 - ⇒ Implémentation par SUN (ONC RPC --Open Network Computing RPC)
 - ↳ V1 en 1989 (RFC 1057), V2 en 1995 (RFC 1831)
- ❑ Le client demande une procédure (service) située sur un site distant (masquage de la communication réseau)
 - ⇒ Les paramètres sont empilés sur le canal de communication plutôt que dans la mémoire

Pourquoi utiliser les RPC?

- ✓ Facilité de programmation.
- ✓ Sujet bien maîtrisé de nos jours.
- ✓ Facilité d'adaptation d'un code existant.
- ✓ Possibilité de générer le code de l'appelant et de l'appelé automatiquement.
- ✓ RPC est à la base de l'implémentation du NFS
- ✓ RPC est la base de l'invocation de méthodes sur des objets distants –RMI

Objectifs et problèmes des RPCs

❑ Objectifs attendus

- ⇒ Forme et effet identiques à ceux d'un appel local
 - ↳ Applications non modifiées lors du passage à l'appel distant
 - ↳ Mise au point locale avant répartition
 - ↳ Simplicité d'utilisation
- ⇒ Abstraction
 - ↳ Indépendance vis-à-vis des protocoles de communication
 - ↳ Réutilisation

❑ Problèmes

- ⇒ Sémantique complexe en cas de panne
 - ↳ Les processus client et serveur peuvent tomber en panne indépendamment
 - ↳ Le réseau introduit une incertitude
- ⇒ Restrictions sur les paramètres
 - ↳ Passage de structures complexes, possible mais compliqué

Comment ça marche les RPCs?

1. Interface pour les procédures distantes

- ⇒ **IDL** (Interface Definition Language/interface description language)

2. Compilation d'IDL

3. Résultat de compilation

- ⇒ **Talon client** (souche/stub) et **talon serveur** (squelette) pour chaque procédure définie
- ⇒ Interface entête (headers)

Langage de description d'interface (IDL)

Interface = “contrat” entre client et serveur

- ❑ Définition commune abstraite

- ⇒ Langage neutre (adapté à des langages multiples), proche du C++
- ⇒ Indépendante de la représentation des types
- ⇒ Indépendante de la machine → **masquer les divers problèmes d'hétérogénéité**

- ❑ Contenu minimal

- ⇒ Identification des procédures (nom, version)
- ⇒ Définition des types des paramètres, résultats, exceptions
- ⇒ Définition du mode de passage (IN, OUT, IN-OUT)

- ❑ Extensions possibles

- ⇒ Procédures de conversion pour types complexes
- ⇒ Propriétés non fonctionnelles (qualité de service), non standard

Langage de description d'interface -IDL (suite)

- ❑ Pas de standard, mais s'appuie sur un langage existant

- ⇒ IDL RPC -RPCL (proche du C)
- ⇒ Java et C# définissent leur propre IDL
- ⇒ CORBA sur C++

Souches et squelettes

❑ A partir d'une spécification d'interface, on peut générer une **souche**:

⇒ Emballage des paramètres et envoi de la requête

⇒ Attente et déballage du résultat

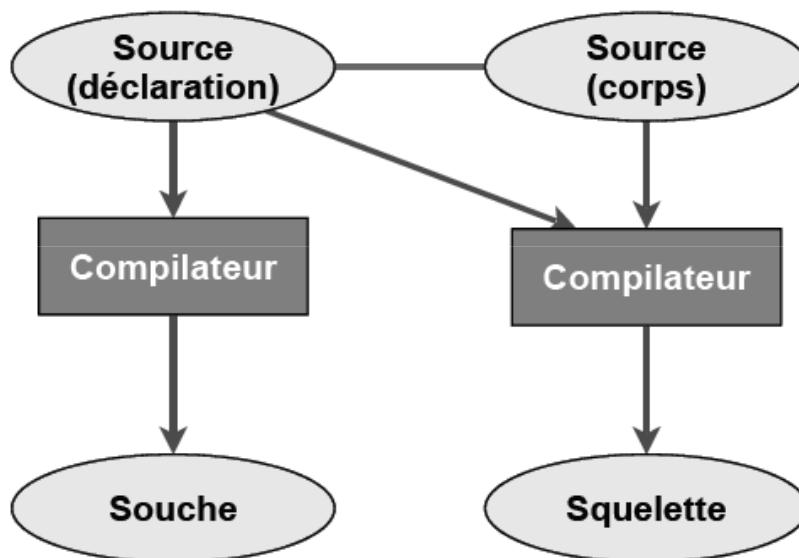
❑ On peut également générer un **squelette**:

⇒ Réception de la requête et déballage des arguments

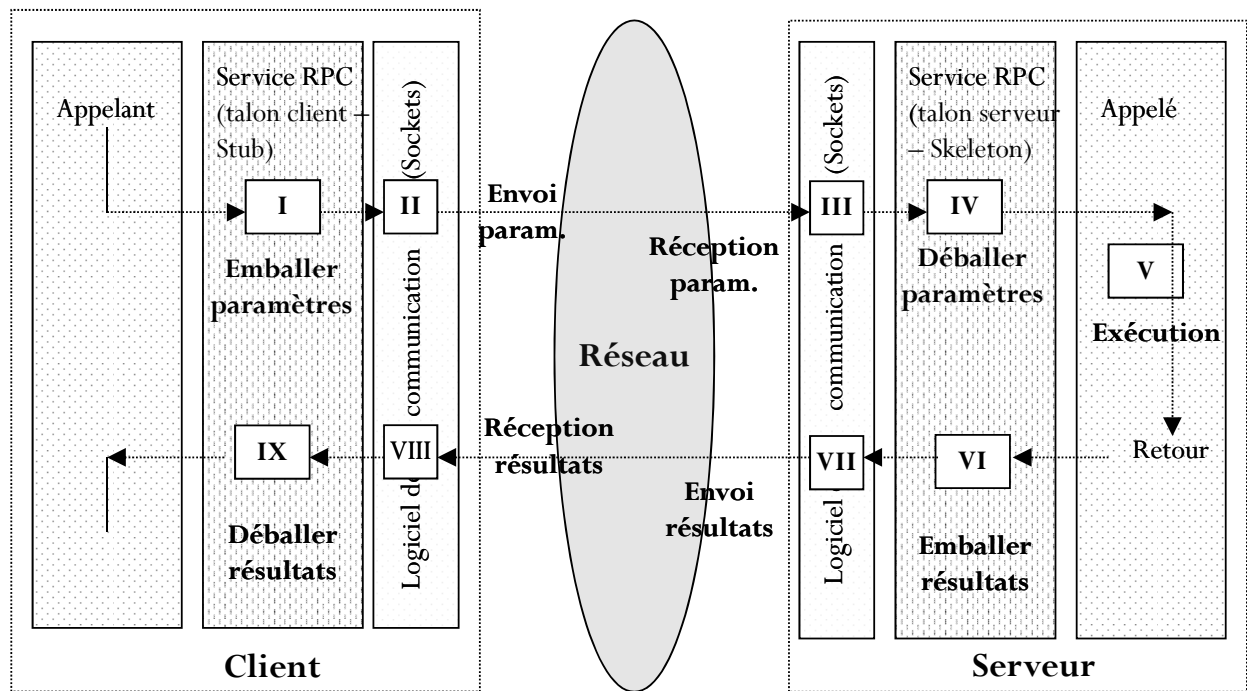
⇒ Appel du sous-programme réel

⇒ Emballage et renvoi du résultat

Souches et squelettes (suite)



RPC: Principe de réalisation



RPC: Rôle des souches

Talon client: Stub

- ❑ *Procédure d'interface sur le site client (représentant du serveur sur le site client)*
 - ⇒ Reçoit l'appel en mode local
 - ⇒ 'emballe' les paramètres et transforme en appel distant (envoi message)
 - ⇒ Blocage du processus client: attend les résultats en provenance du serveur
 - ⇒ Reçoit et 'déballer' les paramètres résultats, et les retourne au programme client 'comme' dans un retour de procédure local
- ⇒ **Liaison initiale**

Talon serveur: Skeleton

- ✓ *Procédure d'interface sur le site serveur (représentant du client sur le site serveur)*
 - ⇒ Reçoit l'appel sous forme de message
 - ⇒ 'déballer' les paramètres d'appel
 - ⇒ Exécution de la procédure du service invoquée sur le site serveur
 - ⇒ 'emballe' les paramètres résultats, et les transmet par message
- ⇒ **Enregistrement initiale**

RPC: Génération des talons (1/3)

❑ Principe

- ⇒ La structure des talons est bien définie et ils peuvent être créés automatiquement
- ⇒ Afin d'éviter à écrire des filtres XDR et de programmer la fonction d'aiguillage du serveur, le système propose un utilitaire d'aide au développement de programmes utilisant les RPC
 - ⇒ Cet outil s'appelle ***rpcgen*** (cf. ***man rpcgen***)
- ⇒ Il s'agit d'un pré-compilateur qui engendre plusieurs fichiers à partir d'une description synthétique du programme donné dans un fichier dont le type est ".x"

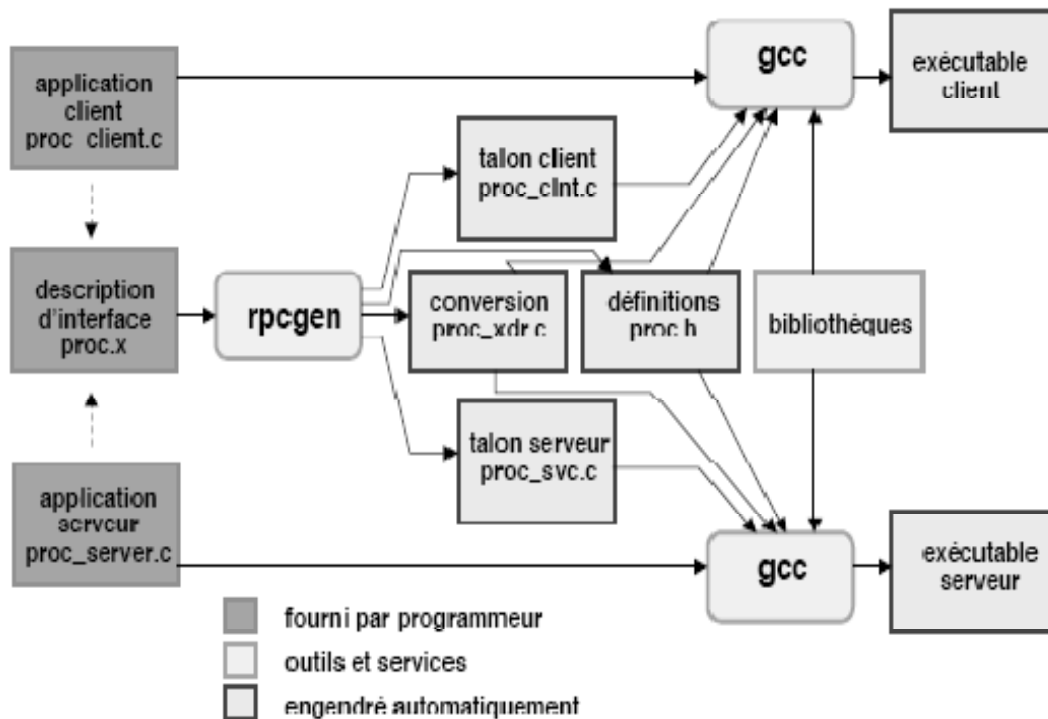
❑ Mise en œuvre

- ⇒ Les informations nécessaires dépendantes de l'application sont décrites dans un IDL

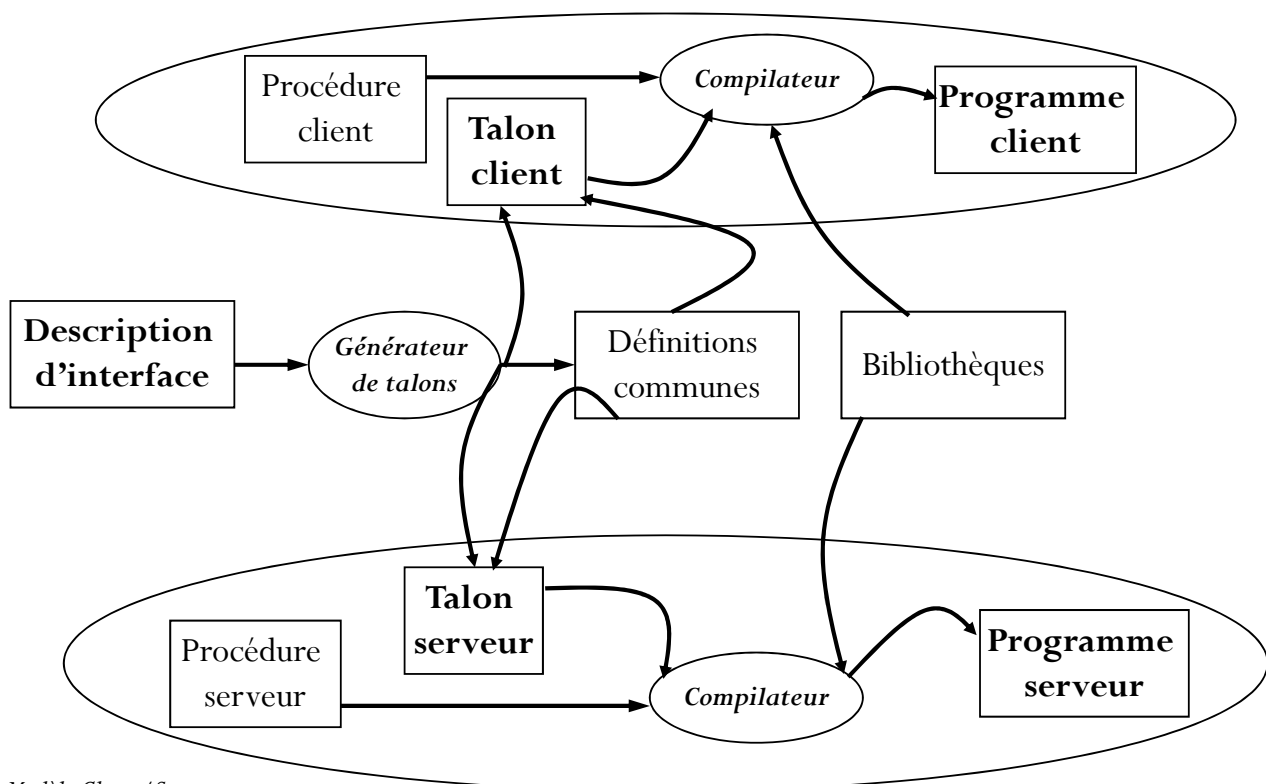
RPC: Génération des talons (2/3)

- ❑ Si le fichier de description s'appelle ***fichier.x***, les fichiers produits seront les suivants:
 - ⇒ Un fichier à inclure dans les codes du serveur et des clients (***fichier.h***)
 - ⇒ Un squelette du code du serveur (***fichier_svc.c***)
 - ⇒ Les procédures réalisant les appels distants pour les clients (***fichier_clnt.c***)
 - ⇒ Les filtres xdr (***fichier_xdr.c***)
- ❑ Il reste donc à écrire deux fichiers:
 - ⇒ Un fichier contenant le code des procédures fournies par le serveur qui complètera ***fichier_svc.c***, lui-même contenant déjà la fonction main
 - ⇒ Le code du client qui se servira de ***fichier_clnt.c***, ici on écrit le main

RPC: Génération des talons (3/3)



IDL: Chaîne de compilation



Appel distant vs. appel local

❑ Objectifs

- ⇒ Préserver la sémantique actuel de l'appel de procédure
- ⇒ "transparence" de la distribution: appel "identique" à celle d'un appel local

❑ Objectifs difficiles à atteindre!

- ⇒ Réalisation peu conviviale (même avec l'utilisation d'un IDL)
- ⇒ Sémantique différente de l'appel de procédure même en absence de panne:
 - Exemple: passage de paramètres par référence

➔ Sémantique précise est en fonction de l'implantation du RPC

RPC: Quelques problèmes de mise en œuvre

- ❑ Comment le client connaît-il l'adresse du serveur?
- ❑ Comment fonctionne la transmission des paramètres et résultats
 - ⇒ Mode de passage (valeur, référence?)
 - ⇒ Représentation, emballage/déballage
 - ⇒ Gestion de l'hétérogénéité
- ❑ Traitement des défaillances
 - ⇒ Panne du client ou du serveur et/ou congestion du réseau
- ❑ Désignation et liaison
- ❑ Problèmes de sécurité
 - ⇒ Authentification du client et du serveur
 - ⇒ Confidentialité des échanges
- ❑ Aspects pratiques
 - ⇒ Adaptation à des environnements d'utilisation variables (protocoles, langages, matériels)

RPC: Désignation et liaison

❑ Problèmes:

⇒ Objets à désigner: le site serveur, la procédure appelée

❑ Propriétés souhaitées:

⇒ Désignation globale indépendante de la localisation

↳ Possibilité de reconfiguration des services (panne, régulation de charge,...)

❑ Moment de liaison

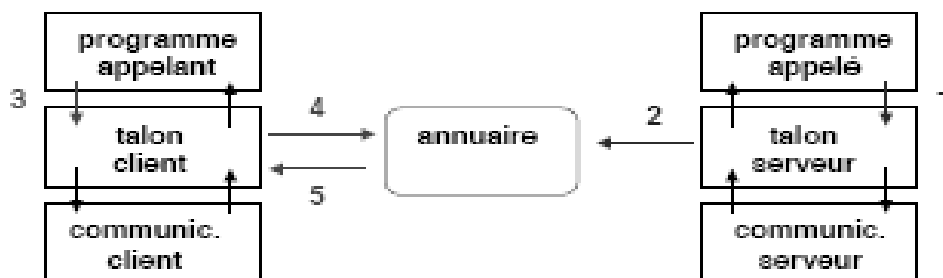
⇒ Statique: localisation du serveur connu à la compilation

⇒ Dynamique: localisation non connue à la compilation mais déterminé à l'exécution (au premier appel RPC)

↳ Désignation symbolique des services (non liée à un site d'exécution)

↳ Possibilité d'implémentation retardée

RPC: Désignation et liaison utilisant un annuaire



❑ 1, 2: le serveur s'enregistre auprès de l'annuaire <nom, adres_serveur, port>

❑ 3, 4, 5: le client consulte l'annuaire pour trouver <adresse_serveur, port> à partir de <nom>

❑ L'appel peut alors avoir lieu

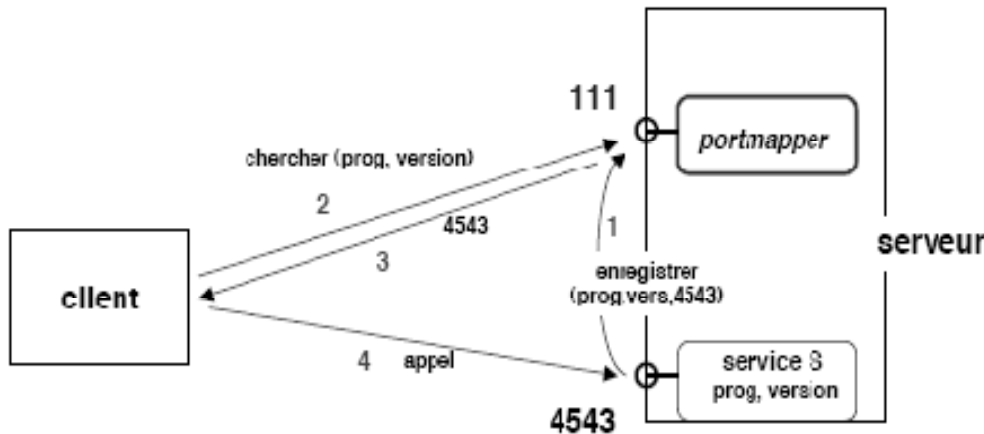
❑ Exemples:

⇒ DNS en Internet

⇒ X500 en DCE

RPC: Désignation et liaison utilisant le portmapper

- ❑ Si le serveur est connu (cas fréquent): on peut utiliser un service de nommage local au serveur, le *portmapper*
- ❑ Un service enregistre le n° de port de son veilleur auprès du *portmapper* et le veilleur se met en attente sur ce port
- ❑ Le *portmapper* a un n° de port fixé par convention (111)



Modèle Client / Serveur

48

RPC: Représentation des paramètres

- ❑ Problème: conventions différentes de représentation des données
 - ⇒ Conventions de représentation des nombres flottants
 - ⇒ Conventions de représentation des structures complexes
 - ⇒ N'utilisent pas le même système de codage
 - ⇒ Utilisent des formats internes différents (pour les types de base)
- ❑ Solution classiquement placée dans la couche 6 du modèle OSI
- ❑ Solution normalisée ASN.1 (Abstract Syntax Notation)
 - ⇒ Syntaxe abstraite de transfert de données, norme OSI de définition universelle des types
- ❑ Autres solutions
 - ⇒ Protocoles propriétaires:
 - ↳ Sun XDR utilisé dans NFS
 - ↳ NIDL (Network IDL) de HP
 - ⇒ Représentation locale pour le client et conversion par le serveur
 - ⇒ Négociation entre client et serveur

Modèle Client / Serveur

49

Un exemple de convention de représentation de paramètres: XDR

❑ **XDR: eXternal Data Representation** (Sun)

- ⇒ Format de transmission sur réseau
- ⇒ Bibliothèque de programmes de conversion

❑ Ensemble de routines disponibles pour les types de base (de C)

- ⇒ Données élémentaires
- ⇒ Structures simples (tableaux, chaînes, unions)
- ⇒ Traitement de pointeurs, gestion de mémoire

❑ Utilisation: programmes d'emballage/déballage

- ⇒ Courante: par les générateurs automatiques de talons, dans tous les cas usuels
- ⇒ Avancée: directement par le programmeur, pour le traitement de structures de données complexes

RPC: Gestion des paramètres

❑ Problèmes des paramètres

- ⇒ Les espaces d'adressage de l'appelant et de l'appelé sont distincts
 - ↳ Pas de passage par référence
 - ↳ Les pointeurs perdent leur signification (difficulté pour passer des structures complexes)
- ⇒ Les paramètres sont transmis sur le réseau
 - ↳ Représentation sérialisée des structures
- ⇒ Les machines client et serveur peuvent être hétérogènes
 - ↳ Conversion de format

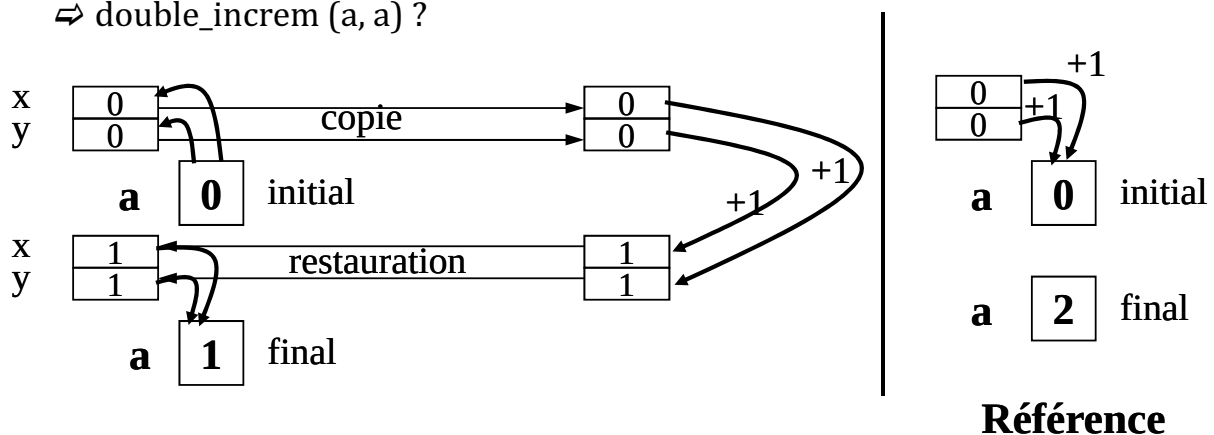
RPC: Passage des paramètres

- ❑ Plusieurs modes de passage sont possibles
 - ⇒ Passage par référence (adresse)
 - ↳ Impossible puisque espaces distincts
 - ⇒ Passage par valeur
 - ↳ Pas de problème
 - ⇒ Passage par copie restauration
 - ↳ A l'appel, copie des valeurs des paramètres de l'appelant vers l'appelé
 - ↳ Au retour, copie des nouvelles valeurs de l'appelé vers l'appelant
- ❑ Solution normalisée:
 - ⇒ IDL de DCE (Distributed Common Environment)
 - ⇒ IDL de CORBA (Common Object Request Broker Architecture)



Passage par copie restauration et passage par référence

- ❑ procédure `double_increm (x, y)`
 - ⇒ $x = x + 1$
 - ⇒ $y = y + 1$
- ❑ Quel est l'effet de:
 - ⇒ $a = 0$
 - ⇒ `double_increm (a, a) ?`



RPC: Sémantique en cas de panne

- ❑ Comment sont traités les erreurs?
- ❑ Détection de panne = expiration de délai de garde
- ❑ En cas d'erreurs, la sémantique de l'appel est variable (dépend des hypothèses de pannes et des mécanismes de reprise mis en œuvre)
 - ⇒ Indéfini (pas d'hypothèses)
 - ⇒ Au plus une fois (0 ou 1 fois)
 - ↳ On évite les exécutions répétées, mais on ne garantit pas le succès
 - ↳ Cas d'opération non idempotente
 - ⇒ Au moins une fois (appel répété, plusieurs exécutions possibles, au moins une réussit)
 - ↳ Si expiration du délais, alors réémission de la requête
 - ↳ Plusieurs exécutions possibles du service
 - ↳ Acceptable si opération idempotente (i.e. $A \circ A = A$, 2 appels successifs ont le même effet que 1 appel, par exemple lecture)
 - ⇒ Exactement une fois (idéale...)
 - ↳ Une seule exécution est effectuée et celle-ci réussit toujours

RPC: Différents types de pannes

- ❑ Pannes franches (des calculateurs)
 - ⇒ Panne du serveur
 - ⇒ Panne du client
 - ↳ Le serveur est dit orphelin (orphan)
- ❑ Pannes d'omission (de messages)
 - ⇒ Il existe des RPC implanté au dessus d'une communication non fiable pour des raisons d'efficacité (exemple Chorus)
 - ⇒ Mécanisme de traitement des perte de messages à prévoir dans la conception du RPC
- ❑ Pannes temporelles
 - ⇒ La réponse ne parvient pas avant une date limite définie par un délai de garde
- ❑ Pannes quelconques
 - ⇒ Possibilité d'utiliser une redondance massive (plusieurs exécutions en parallèle de la même requête)

RPC: Traitement des défaillances

❑ Différents cas de défaillances

⇒ Système de communication

↳ Congestion du réseau, messages retardés

↳ Perte de messages

↳ Altération de messages

⇒ Serveur

↳ Défaillance avant l'exécution de la procédure

↳ Défaillance pendant l'exécution de la procédure

↳ Défaillance après l'exécution de la procédure

↳ Défaillance définitive ou reprise possible

↳ Congestion du serveur

⇒ Client

↳ Défaillance pendant le traitement de la requête

RPC: Performances

❑ Dépendent de 5 parties principales:

① Encapsulation de l'appel

② Le traitement du noyau du client et l'envoi

③ Le transit

④ Réception par le noyau du serveur

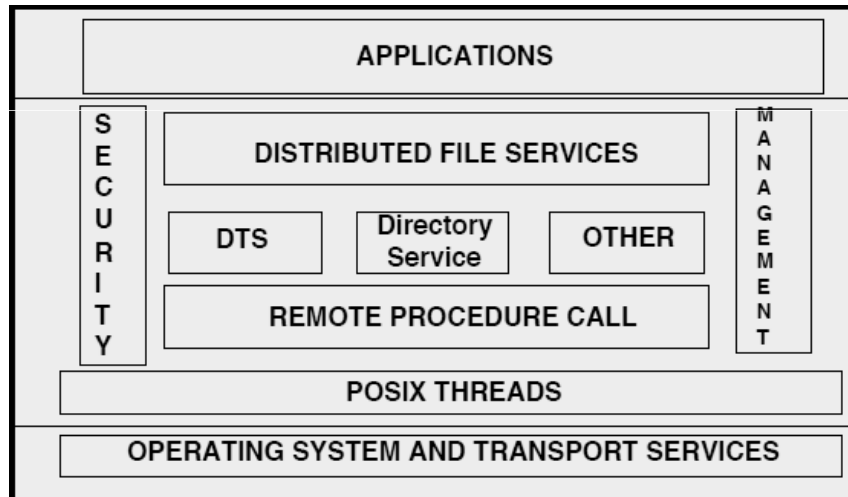
⑤ Décapsulation du message

Environnement Client/Serveur --DCE

❑ **DCE** –Distributed Computing environment

⇒ Environnement complet de programmation d'applications réparties, fondé sur le modèle client/serveur.

❑ **Architecture de DCE:**



Source: <http://www.ts.infn.it/computing/htasc>

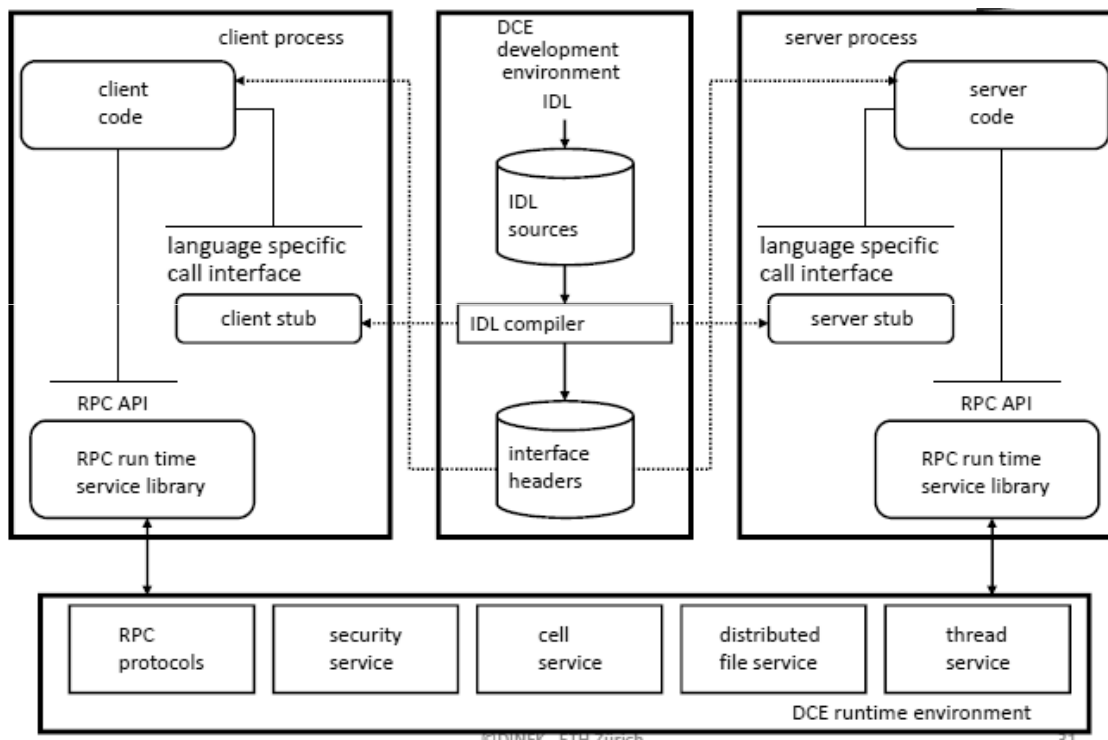
Composants de l'architecture DCE

(Source: <http://www.ts.infn.it/computing/htasc>)

- ❑ **DCE threads:** support the creation, management and synchronization of multiple threads of control within a program
- ❑ **DCE RPC:** facility for calling a procedure on a remote machine as if it was a local procedure call, shielding the details of network communications
- ❑ **DCE Directory Service:** allows to keep track information about users, machines and other resources geographically dispersed
- ❑ **DCE DTS:** Distributed Time Service provides a way to periodically synchronize the clock on the different hosts in a distributed system
- ❑ **DCE DFS:** Distributed File Service is an application built on the underlying DCE services that provides the ability to share data throughout a worldwide system

Client-Serveur au dessus de DCE

(Source: [Alons11])



Modèle Client / Serveur

60

DCE - Atouts et limites

❑ Atouts de DCE:

- ⇒ architecture de type « boîte à outils »
 - ↳ Chaque fonction est utilisable via une interface "normalisée"
 - Intégration à "gros grain"
 - ↳ Standard de fait (diffusion dans la communauté industrielle)
 - Exp. RPC, service de noms, service d'authentification
- » Portabilité etinteropérabilité

❑ Limites de DCE:

- ⇒ Exp. Utilisation des services de threads et de RPC (limitations équivalentes à celles du C/S).
- ⇒ peu d'outils de développement disponibles aujourd'hui.

Modèle Client / Serveur

61

Client-serveur -Conclusions (1/3)

- ❑ Les systèmes répartis sont populaires → il est souhaitable de bien maîtriser!
- ❑ Il existe de nombreuses manières de développer une application répartie
- ❑ Il faut considérer les différents facteurs
 - ⇒ Le système doit-il être sûr?
 - ⇒ Doit-il supporter plusieurs langages?
- ❑ Il est difficile de comparer deux concepts de systèmes répartis complexes

Client-serveur classique—Conclusions (2/3)

- ✓ Cycle de vie lié au serveur
- ✓ Centré IP
- ✓ Structure d'application **statique**:
 - ⇒ Pas de création dynamique de serveur, pas de possibilité de redéploiement entre sites.
- ✓ Basé sur le DNS
- ✓ RPC/RMI
- ✓ Communication **Synchrone**
- ✓ Asymétrique
- ✓ Axé sur des modèles de liaison et d'intégration du langage de programmation (stub IDL/XDR, compilateurs, etc...)
- ✓ Sécurité de type Kerberos : acl, crypto

Conclusions sur les RPCs (3/3)

- ❑ Des mécanismes plus évolués visent à remédier à ces limitations.
- ❑ C/S objet
 - ⇒ RMI: objets répartis de Java
 - ↳ Supporte la migration de code
 - ⇒ CORBA:
 - ⇒ DCOM (Microsoft): concurrent de CORBA
- ❑ XML-RPC
 - ⇒ Utilise des technologies normalisées:
 - ↳ XML, HTTP
 - ↳ Bénéficie d'un niveau de sécurité normalisé: HTTPS (SSL+X509)
- ❑ SOAP (Microsoft)
 - ⇒ Basé sur XML-RPC

Néographie

- ❑ La majorité des transparents de ce cours sont pris de:

[Krako]	http://sardes.inrialpes.fr/~krakowia/Enseignement/M2P-GI/ ; Systèmes et Applications Répartis (Sacha Krakowiak, Grenoble).
[Duvallet]	http://litis.univ-lehavre.fr/duvallet/ ; cours de Systèmes et Applications Réparties, Claude Duvallet
[Riveill99]	M. Rivell, Modele client-serveur, cours ensimag, 1999. ref. 03-99-slides-client-serveur.pdf
[Esnard]	http://www.labri.fr/~esnard ; Programmation des Applications Réparties, Aurélien Esnard, ENSEIRB PG306-IT343
[Quinson08]	Programmation d'Applications Reparties, Martin Quinson, cours ESIAL, 2008.
[Alons11]	Gustavo Alonso, Timothy Roscoe, Donald Kossmann, Networks and Operating Systems, Chap. 3: Remote Procedure Call (RPC). ETH Zurich, 2011